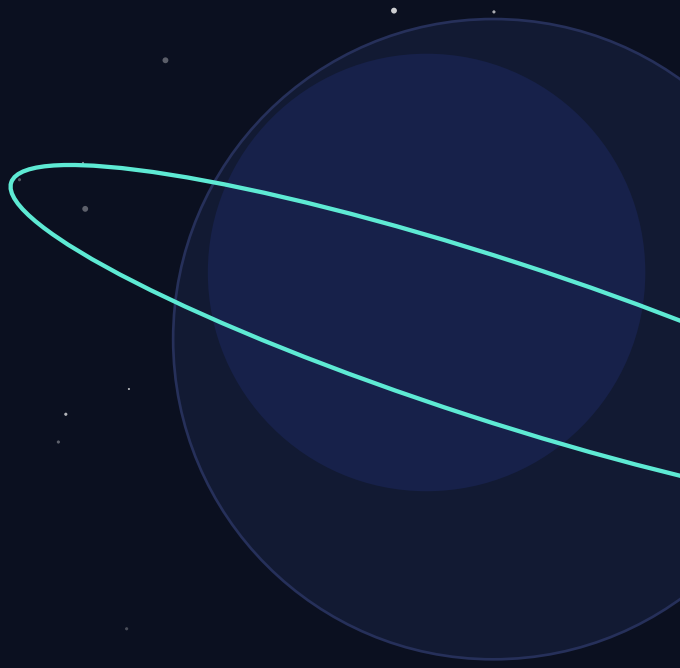


AI Exoplanet Classifier & Habitability Explorer

Complete build plan: data, machine learning, backend, UI and deployment

PROJECT PLAN | OCTOBER 2026

Data: NASA Exoplanet Archive (PSCompPars, Kepler KOI, TESS TOI) and Kaggle mirrors

Stack: Python, scikit-learn / XGBoost, FastAPI, React, Docker

Contents

#	Section	What it covers
1	**Project overview**	What the site does, goals, and scope
2	**System architecture**	How the pieces fit together
3	**Data sources and preparation**	NASA TAP queries, tables, key columns
4	**The classifier model**	Confirmed planet vs false positive
5	**The habitability score**	Name lookup, Earth Similarity Index, habitable zone
6	**Backend design (FastAPI)**	Endpoints, schemas, project structure
7	**Frontend and UI design**	Design system, pages, wireframes
8	**Testing and evaluation**	What to measure and check
9	**Deployment**	Docker, hosting, CI
10	**Timeline**	Four-week schedule with deliverables
11	**Risks and mitigations**	What can go wrong and how to handle it
12	**Extras, documentation and references**	Making the project stand out

1. Project overview

This project is a web application where a visitor can do two things with real NASA exoplanet data. First, they can check whether a detected transit signal is a genuine planet or a false alarm. Second, they can type the name of a known planet and see how suitable it might be for human life. Both answers come from machine learning models trained on public catalogs from the NASA Exoplanet Archive, and the site explains each answer instead of only showing a number.

Goals

- Build a **working, deployed** web tool with a clean, modern interface, not just a notebook.
- Train a **well-evaluated classifier** on Kepler (and optionally TESS) data with calibrated probabilities.
- Provide a **transparent habitability score** that a visitor can understand and challenge.
- Make every prediction **explainable** (which inputs pushed the result up or down).
- Keep the codebase clean enough to present as a portfolio or final-year project.

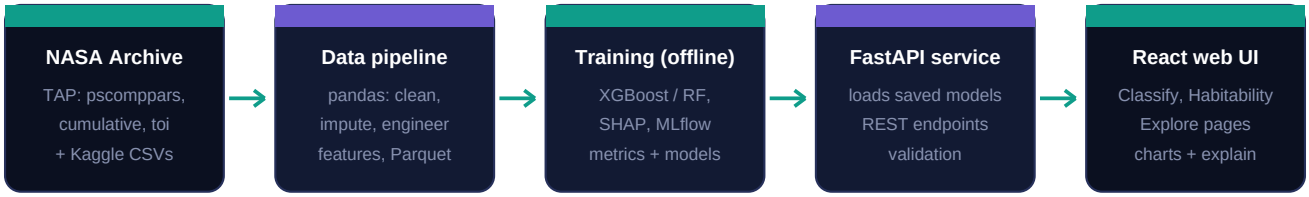
The three tools on the site

Tool	What the user does	What they get back	ML behind it
Classify	Enters stellar and orbital parameters, or picks a sample from a held-out test set	Confirmed Exoplanet vs False Positive, a probability, and the top features behind it	Gradient-boosted trees (XGBoost) or Random Forest, calibrated, with SHAP
Habitability Check	Types a planet name (for example Kepler-452 b)	A 0-100 habitability potential score, a component breakdown, and a comparison with Earth	Score built from the Earth Similarity Index and habitable-zone physics, learned by a regressor that also handles missing data
Explore	Browses charts and model information	Dataset statistics, model metrics, feature importance, honest limitations	No model; reads saved metrics and the planets table

Scope note: catalog data, not raw light curves
The classifier works on catalog-level measurements (orbital period, transit depth, stellar temperature and so on). These are the tabular outputs of the Kepler and TESS pipelines. It does not analyse raw telescope light curves. This keeps the project realistic for about a month of work. Light-curve analysis is listed as an optional extension in section 12.

2. System architecture

The guiding rule is **train offline, serve online**. All heavy work (downloading data, cleaning, training, evaluating) happens in scripts and notebooks inside an *ml/* folder. Those scripts write small files: a trained model, the list of features it expects, a metrics file, and a cleaned planets table. The web API only loads those files and answers requests. This makes the site fast, cheap to host, and easy to test.



Offline: data + training produce files (model, feature list, metrics, planets table).
Online: the API only loads those files, so it starts fast and stays lightweight.

How a request flows

Classify

1. The user fills the form (or clicks a sample) and the React app sends the values to *POST /api/classify*.
2. FastAPI validates the values (types and sensible ranges) with Pydantic.
3. The service arranges them in the exact feature order the model was trained on and applies the same transforms (for example log scaling).
4. The model returns a probability, and SHAP values explain the top contributing features.
5. The UI shows the label, the probability gauge, and a bar chart of feature impacts.

Habitability by name

1. The user types a name; the UI calls */api/search* for autocomplete suggestions as they type.
2. On submit, the backend normalises the name, finds the planet in the PSCompPars table (fuzzy matching if needed), and loads its parameters.
3. Missing values are estimated (and flagged), then the score components and the regressor output are computed.
4. The UI shows the score, the band (for example Promising), the breakdown, a habitable-zone diagram, and a comparison with Earth.

Suggested repository layout

```
exoplanet-ai/
|-- ml/                      # offline work
|   |-- notebooks/           # EDA and experiments
|   |-- train_classifier.py
|   |-- train_habitability.py
|   `-- features.py          # shared feature code (used by training AND the API)
|-- backend/
|   `-- app/
|       |-- main.py          # FastAPI app, CORS, startup model loading
|       |-- routers/         # classify.py, habitability.py, meta.py
|       |-- schemas.py       # Pydantic request/response models
|       |-- services/        # model_service.py, lookup_service.py, scoring.py
|       `-- artifacts/       # model.joblib, features.json, metrics.json, planets.parquet
|-- frontend/                # React + Vite + Tailwind
|-- scripts/fetch_data.py    # downloads and caches NASA tables
|-- tests/
`-- docker-compose.yml
```

Why share features.py?
A very common bug is *training/serving skew*: the model is trained with one set of transforms and the API accidentally applies a slightly different set. Putting the transforms in one shared module that both training and the API import removes this whole class of problems.

3. Data sources and preparation

The primary source is the **NASA Exoplanet Archive**, which exposes its tables through a TAP (Table Access Protocol) web service. You send an ADQL/SQL-style query and get CSV back, so there is no login and no manual download. Kaggle mirrors of the same data are useful as a quick start or as a fallback if the service is unavailable, but the official service is fresher and has clear provenance.

Table (TAP name)	What it contains	Used for
pscomppars Planetary Systems Composite Parameters	One row per confirmed planet (6,000+), with the best-available planet and host-star parameters merged into a single row	Name lookup, habitability score, Explore charts
cumulative Kepler KOI table	Roughly 9,500 Kepler Objects of Interest, each labelled CONFIRMED, CANDIDATE or FALSE POSITIVE, with transit and stellar measurements	Primary training data for the classifier
toi TESS Objects of Interest	TESS candidates with their follow-up dispositions (for example CP/KP confirmed, PC candidate, FP false positive)	Optional second classifier or a mission toggle

Fetching and caching the data

Write one script, *scripts/fetch_data.py*, that downloads each table once and saves it as Parquet in *data/*. Everything else reads the cached files, so training does not depend on the network. Commit a small snapshot (or attach it to a release) so the project still works if the archive changes.

```
import pandas as pd
import urllib.parse as up

BASE = "https://exoplanetarchive.ipac.caltech.edu/TAP/sync"

def tap(query: str) -> pd.DataFrame:
    url = f"{BASE}?query={up.quote_plus(query)}&format;=csv"
    return pd.read_csv(url)

pscomp = tap("select * from pscomppars")
koi    = tap("select * from cumulative")
toi    = tap("select * from toi")

pscomp.to_parquet("data/pscomppars.parquet")
koi.to_parquet("data/koi.parquet")
toi.to_parquet("data/toi.parquet")
```

Key columns

Purpose	Columns (Kepler KOI)	Columns (PSCompPars)
Label / name	`koi_disposition`	`pl_name`, `hostname`
Orbit and transit	`koi_period`, `koi_duration`, `koi_depth`, `koi_impact`	`pl_orbper`, `pl_orbsmax`
Planet	`koi_prad`, `koi_teq`, `koi_insol`	`pl_rade`, `pl_bmasse`, `pl_dens`, `pl_eqt`, `pl_insol`
Detection quality	`koi_model_snr`	-
Host star	`koi_steff`, `koi_srad`, `koi_slogg`	`st_teff`, `st_rad`, `st_mass`, `st_lum`, `st_met`

Note: in PSCompPars, `st_lum` is stored as log10(L / L_{Sun}), so convert it before using it in physical formulas. Always check units in the archive's column documentation before you compute anything.

Cleaning steps

1. Keep only the columns you need; rename them to friendly names in one place (a dictionary in *features.py*).
2. Drop rows with too many missing features; for the rest, either let XGBoost handle NaN natively or impute with the median inside a scikit-learn pipeline.
3. Remove impossible values (negative radii, periods of zero) and check extreme outliers by eye before deciding to cap them.
4. Log-transform strongly skewed columns (period, depth, radius, SNR) so tree and neural models behave well.
5. Save the cleaned tables as Parquet and record the download date and row counts in a small *data_manifest.json*.

4. The classifier model

Task. Given measurements of a transit signal and its host star, predict whether it is a confirmed planet or a false positive. Train on Kepler KOI rows labelled CONFIRMED (class 1) or FALSE POSITIVE (class 0). Rows labelled CANDIDATE are still unresolved, so they are **excluded from training**. This is useful: after training you can score the candidates and show the user a ranked list of the most promising ones.

Features

Group	Features	Why they help
Transit shape	period, duration, depth, impact parameter	False positives such as eclipsing binaries often show deeper, odd-shaped or inconsistent transits
Detection strength	model signal-to-noise (SNR)	Weak signals are far more likely to be noise or artefacts
Planet size and heat	radius, equilibrium temperature, insolation	Implausibly large radii are a classic sign of a stellar companion rather than a planet
Host star	effective temperature, radius, surface gravity (log g)	Planet radius is derived from stellar radius, so errors in the star propagate into the planet

Avoid data leakage (important)
The Kepler table contains columns such as `koi_fpflag_nt`, `koi_fpflag_ss`, `koi_fpflag_co` and `koi_fpflag_ec`. These are *outputs of the vetting process* that helped decide the label. If you feed them to the model you will see 99% accuracy that means nothing. Leave them out, and also leave out the `koi_pdisposition` and score columns. Also split train and test **by host star** (`kepid`), so planets of the same star never appear on both sides.

Training procedure

1. **Split:** hold out a final test set (for example 15%) using *GroupShuffleSplit* on the star ID; never touch it until the end.
2. **Baseline:** train a Random Forest first. It needs little tuning and gives a number to beat.
3. **Main model:** train XGBoost (or LightGBM) and tune depth, learning rate, number of trees and subsampling with *GroupKFold* cross-validation. Log every run in MLflow.
4. **Optional:** a small neural network (MLP) in PyTorch or TensorFlow for comparison. On tabular data it rarely beats boosted trees, which is itself a worthwhile finding to report.
5. **Calibrate:** fit isotonic or sigmoid calibration on a validation split so that a displayed 90% really means about 90%.
6. **Explain:** compute SHAP values for the final model, for both global importance and per-prediction explanations.
7. **Freeze:** save the model, feature list, threshold and metrics to *artifacts/* with a version string.

```
from sklearn.model_selection import GroupShuffleSplit
from xgboost import XGBClassifier
from sklearn.metrics import roc_auc_score, average_precision_score, f1_score

df = koi[koi.koi_disposition.isin(["CONFIRMED", "FALSE POSITIVE"])].copy()
X = df[FEATURES]                                # FEATURES defined in ml/features.py
y = (df.koi_disposition == "CONFIRMED").astype(int)

gss = GroupShuffleSplit(n_splits=1, test_size=0.15, random_state=42)
train_idx, test_idx = next(gss.split(X, y, groups=df.kepid))

model = XGBClassifier(n_estimators=400, max_depth=5, learning_rate=0.05,
                      subsample=0.8, colsample_bytree=0.8, eval_metric="logloss")
model.fit(X.iloc[train_idx], y.iloc[train_idx])

p = model.predict_proba(X.iloc[test_idx])[:, 1]
print("ROC-AUC", roc_auc_score(y.iloc[test_idx], p))
print("PR-AUC ", average_precision_score(y.iloc[test_idx], p))
print("F1     ", f1_score(y.iloc[test_idx], p > 0.5))
```

Models to compare

Model	Role	Notes
Random Forest	Baseline	Robust, little tuning; needs imputation for missing values
XGBoost / LightGBM	Likely winner	Handles missing values natively, strong on tabular data, works well with SHAP
MLP neural network	Optional comparison	Needs scaling and more tuning; report honestly if it does not win

How to evaluate

- **ROC-AUC and PR-AUC** (PR-AUC matters because the classes are not perfectly balanced), plus precision, recall and F1 at your chosen threshold.
- **Confusion matrix** on the held-out test set, and a **calibration curve** before and after calibration.
- **Error analysis:** look at the false positives and false negatives. What do they have in common (low SNR, giant radii, certain star types)?
- **Candidate ranking:** score the CANDIDATE rows and show the top ones. Check that a few later-confirmed ones rank high if the data allows.

5. The habitability score

This is the second feature: the user types a planet name and the system decides how good that planet might be for humans. It is the most interesting part scientifically, and it needs one honest decision up front.

> **There is no ground-truth label for habitability**
> No catalog tells us which planets are truly habitable; we have only one example of life, Earth. So a model cannot simply be trained on a "habitable" column. The honest approach is to define a **transparent, physics-based score** from published science, and call the result a **habitability potential score**. State this clearly on the site. It measures similarity to Earth-like conditions using measured parameters, not proof of life or a verdict on whether humans could live there.

Step 1: look up the planet by name

1. Normalise the text: lower-case, remove extra spaces and hyphens so "Kepler-452b", "kepler 452 b" and "Kepler-452 b" match.
2. Try an exact match against `pl_name`. If that fails, use fuzzy matching (the `rapidfuzz` library) with a score cut-off of around 80 and return the top five suggestions as "Did you mean...?".
3. If the user types a star name (a `hostname`), list the planets of that system.
4. If nothing matches, offer a manual-entry form that reuses the same parameters, so the tool is never a dead end.

Step 2: compute the components of the score

(a) Earth Similarity Index (ESI). ESI compares a planet with Earth on a few properties using the formula below, where x is the planet's value, x_0 is Earth's value, w is a weight exponent and n is the number of properties combined.

$$ESI_x = \left(1 - |x - x_0| / (x + x_0) \right)^{w / n}$$

Property	Earth reference	Weight w	How to get it
Radius	1.0 R_{Earth}	0.57	`pl_rade` directly
Density	1.0 (5.51 g/cm^3)	1.07	Mass / radius^3 in Earth units, or `pl_dens`
Escape velocity	1.0 (11.19 km/s)	0.70	Square root of (mass / radius) in Earth units
Temperature	255 K equilibrium (288 K surface)	5.58	Use `pl_eqt` with Earth's 255 K equilibrium value so you compare like with like

In the commonly used form, **interior ESI** combines radius and density, **surface ESI** combines escape velocity and temperature, and the global ESI is the geometric mean of the two. Slightly different formulations exist, so write down exactly which one you implement in the README.

(b) Habitable-zone factor. The habitable zone is the range of orbits where liquid water could exist on a planet's surface. Use the insolation flux S (energy received, in Earth units). If `pl_insol` is missing, compute S = L / a^2 with luminosity L in solar units and orbital distance a in AU. As a starting point, a conservative zone is roughly S = 0.36 to 1.1 and an optimistic one roughly S = 0.32 to 1.8 (based on Kopparapu et al., 2013, which varies with star temperature). Give a factor of 1.0 inside the conservative zone and let it fall smoothly to 0 outside the optimistic limits.

(c) Rocky-size factor. Planets larger than roughly 1.6 Earth radii are usually gas-rich rather than rocky, so they are poor candidates for human-style habitability. Use a factor near 1 for radii up to about 1.6, a gentle decline up to about 2.0, and a very low value beyond that.

(d) Stellar suitability factor. Very hot stars are short-lived, and very small red dwarfs are often strongly flaring. A simple factor based on effective temperature (best for Sun-like stars, reduced at both extremes) is enough for this project. Document the thresholds you choose.

Step 3: combine into one score

```
score = 100 * ( 0.40 * ESI
              + 0.35 * HZ_factor
              + 0.15 * rocky_factor
              + 0.10 * star_factor )
```

These weights are a **starting point**, not a scientific constant. Tune them by checking the ranking against known cases, and report the choices in your write-up. Suggested bands:

Score	Band	Typical meaning
80-100	Earth-like potential	Rocky size, in the habitable zone, Earth-like density and temperature
60-79	Promising	Mostly favourable, with one or two weaker factors or uncertain inputs
40-59	Marginal	Some Earth-like traits but a clear problem such as size or flux
20-39	Unlikely	Conditions far from Earth's
0-19	Hostile	For example hot Jupiters or planets far outside the habitable zone

Sanity check: a good score formula passes simple tests. Well-known potentially habitable planets (for example Kepler-452 b, TRAPPIST-1 e and Proxima Centauri b) should land in the upper bands, while hot Jupiters should land near zero. Also compare your ranking with the Habitable Exoplanets Catalog from the Planetary Habitability Laboratory, which publishes its own ESI values.

Step 4: where machine learning comes in

Because the target score is computed from a formula, the ML model's job is not to discover habitability. Its real value is practical:

- Train an XGBoost **regressor** to predict the score from the raw parameters, including when some inputs are missing. Many planets lack a measured mass, so the model learns to produce a reasonable estimate instead of failing.
- Where mass is missing, estimate it from radius using a mass-radius relation, and add the field to an **estimated_fields** list so the UI can tell the user which inputs were not measured.
- Use quantile regression or bootstrapped models to give a **score range** (for example 62-78) when several inputs are estimated.
- Use SHAP so the "Why this score?" panel can show which parameters helped or hurt.
- Optionally train a classifier on the PHL catalog's habitability classes as an independent cross-check of your score.

> **Be upfront about this in the report**
> Say clearly that the regressor reproduces and generalises a physics-based score. Examiners and reviewers respect this honesty far more than a claim that the model "discovered" habitability.

6. Backend design (FastAPI)

FastAPI is a good fit: it is fast, validates input with Pydantic, and generates interactive API documentation at */docs* automatically, which is handy for demos and testing.

Method and path	Purpose	Returns
`POST /api/classify`	Classify one set of parameters	label, probability, top feature impacts, model version
`GET /api/samples`	Random candidates from the held-out test set	A list of parameter sets (optionally with true label hidden)
`GET /api/search?q=`	Autocomplete for planet names	Up to 8 matching planet names
`GET /api/habitability?name=`	Habitability score for a named planet	score, band, components, estimated fields, planet and star parameters
`POST /api/habitability`	Score for manually entered parameters	Same shape as the GET response
`GET /api/model-info`	Metrics, feature importances, dataset stats	Data for the Explore page
`GET /api/health`	Health check for hosting platforms	status and model versions

Example exchange (illustrative values)

```
POST /api/classify
{
  "mission": "kepler",
  "orbital_period": 9.48,       "transit_duration_hr": 2.9,
  "transit_depth_ppm": 615,     "impact_parameter": 0.15,
  "planet_radius_re": 2.26,     "equilibrium_temp_k": 793,
  "snr": 35.8,                  "star_teff_k": 5455,
  "star_radius_rsun": 0.93,     "star_logg": 4.47
}

200 OK
{
  "label": "CONFIRMED",
  "probability": 0.93,
  "threshold": 0.5,
  "top_features": [ {"feature": "snr", "impact": 0.31},
                    {"feature": "planet_radius", "impact": 0.12} ],
  "model_version": "xgb-kepler-v1"
}
```

Backend implementation notes

- **Load once:** load the model, feature list and planets table in a FastAPI lifespan/startup handler, not per request.
- **Validate:** use Pydantic with *Field(gt=0, lt=...)* bounds so absurd inputs return a clear 422 error instead of a strange prediction.
- **Fail gracefully:** unknown planet returns 404 with suggestions; model not loaded returns 503 with a readable message.
- **CORS:** allow only the frontend's origin in production.
- **Performance:** the models are small, so predictions take milliseconds; cache the planet lookup index in memory.
- **Tests:** use pytest with FastAPI's TestClient for every endpoint, including bad-input cases.

7. Frontend and UI design

The goal is a **clean, calm interface** that makes the science approachable. A dark, deep-space theme suits the topic, but keep it restrained: a near-black background, one accent colour, lots of whitespace, and no heavy animation that slows the page.

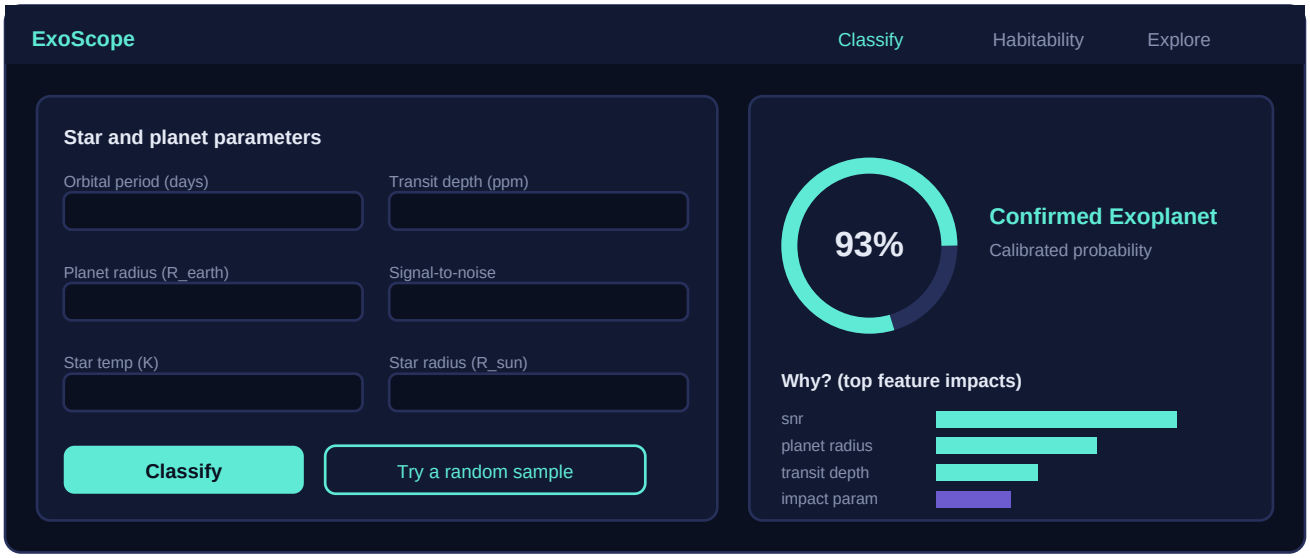
Design system

Element	Choice	Notes
Background / surface	#0B1020 / #121A33	Cards sit on a slightly lighter surface with a thin border
Primary accent	#5EEAD4 (teal)	Buttons, score ring, active nav item, links
Secondary accent	#8B5CF6 (violet)	Charts and secondary highlights only
Text	#E6EAF5 and #8892B0	Main text and muted labels; check contrast is at least 4.5:1
Status colours	#34D399 / #F87171	Confirmed vs false positive; habitable-zone bands
Typography	Inter (or system sans-serif)	Two weights; large numbers for scores; tabular figures in tables
Shape and spacing	12-16 px radius, 8 px spacing grid	Consistent padding keeps it tidy

Tech choices

- **React + Vite** for the app and **Tailwind CSS** for fast, consistent styling.
- **Recharts** (or Plotly) for the gauge, feature-impact bars and the habitable-zone diagram.
- **React Router** for the three pages; **TanStack Query** (or plain fetch hooks) for API calls, loading and error states.
- Optional: Framer Motion for small transitions such as the score ring filling in.

Page 1: Classify



A two-column layout. The form is grouped into **Star** and **Planet** sections with units and short tooltips explaining each term. A "Try a random sample" button fills the form instantly, which makes the demo easy. The result card shows a probability ring, the label, and a "Why?" bar chart of feature impacts.

Page 2: Habitability Check



A large search bar with autocomplete, then three cards: the score ring and band, the "Why this score?" breakdown with the habitable-zone position, and an Earth-vs-planet comparison. A small note shows which inputs were estimated and links to the "How the score works" explanation.

Page 3: Explore and About

- Charts of the dataset (planets by discovery year, radius vs period scatter, habitable-zone overview).
- Model card: data used, metrics, confusion matrix, calibration curve, known limitations.
- "How it works" section in plain language and links to the NASA Exoplanet Archive.

UX details that make it feel finished

- Loading skeletons, friendly error messages, and an empty state with example planets to click.
- Inline validation with units and typical ranges under each input.
- Fully responsive layout (stack the columns on phones) and keyboard-friendly focus styles.
- A short, visible disclaimer: scores are model estimates for learning and exploration, not scientific verdicts.

8. Testing and evaluation

Area	What to check	Tool
Data	Row counts, missing-value rates, units, no duplicated stars across train and test	pandas, a small validation script
Classifier	ROC-AUC, PR-AUC, F1, confusion matrix, calibration curve, error analysis	scikit-learn, matplotlib
Habitability	Ranking sanity tests on known planets; correlation with PHL ESI values; behaviour with missing inputs	pytest, pandas
API	Valid and invalid payloads, unknown names, response schema, response time	pytest + TestClient
Frontend	Form validation, loading and error states, mobile layout	Vitest / manual checklist
End to end	Full flow in Docker Compose before deployment	Manual walkthrough, optional Playwright

9. Deployment

- **Containers:** one Dockerfile for the backend (Python slim image with only the serving dependencies) and one for the frontend build. *docker-compose.yml* runs both locally with one command.
- **Hosting options:** deploy the backend as a Docker service on Render, Railway or a Hugging Face Space; host the built frontend on Vercel or Netlify. Set the API base URL with an environment variable.
- **Free-tier caveat:** free services often sleep when idle, so the first request can be slow. Keep the model small, load it at startup, add a health endpoint, and show a friendly loading state.
- **Artifacts:** keep model files small (well under 100 MB) and version them (for example xgb-kepler-v1) so you can roll back.
- **CI:** a GitHub Actions workflow that runs the tests and builds both images on every push.
- **Config:** never commit secrets; use environment variables for any keys, allowed origins and model paths.

10. Timeline (about four weeks)

Week	Focus	Main tasks	Deliverable
1	Data and baseline	Write fetch_data.py; explore data; clean and engineer features; split by star; train Random Forest baseline	Cached datasets, EDA notebook, baseline metrics
2	Models	Tune XGBoost; calibrate; add SHAP; build the ESI and habitable-zone score; train the regressor; run sanity tests	Saved models and artifacts, metrics.json, model card draft
3	Backend and core UI	FastAPI endpoints with validation and tests; React project, design system, Classify and Habitability pages	Working app running locally end to end
4	Polish and launch	Explore page; UI polish and responsive fixes; Docker; deploy; README with screenshots; short demo video	Live URL, repository, report or slides

Build the thinnest working version first. By the end of week 2, a command-line script should already classify a sample and score a planet. The website then becomes a presentation layer on top of working models, which removes most of the schedule risk.

11. Risks and mitigations

Risk	Why it matters	Mitigation
Data leakage inflates accuracy	Vetting flags are derived from the label	Exclude those columns; split by host star; report PR-AUC and error analysis
No ground truth for habitability	Score could be mistaken for a scientific verdict	Transparent formula, validation against PHL, clear "potential score" wording and disclaimer
Missing parameters	Many planets lack mass or temperature	Estimate, flag estimated fields, show a score range
Name mismatches	Users type names in many formats	Normalisation, fuzzy matching, suggestions, manual-entry fallback
Class imbalance	Accuracy can look good while a class is poorly handled	Use PR-AUC, class weights, and inspect per-class recall
NASA service change or downtime	Training or the site could break	Cache Parquet snapshots; pin column names; keep a Kaggle backup
Free hosting cold starts	Slow first load during a demo	Small models, startup loading, health endpoint, loading UI, warm up before demos
Scope creep	Extras delay the core features	Ship Classify and Habitability first; treat everything else as optional

12. Extras, documentation and references

Ideas that make the project stand out

- **Compare two planets** side by side (for example Earth vs Kepler-452 b).
- **Candidate leaderboard:** the top-ranked unconfirmed Kepler or TESS candidates according to your model.
- **"What if" sliders:** change a planet's radius or star temperature and watch the score move.
- **Model card page** with metrics, limitations and data provenance, in the style used by professional ML teams.
- **Light-curve extension:** use the *lightkurve* library to download real light curves and train a small 1D CNN to classify transits. This is a larger task, so treat it as stretch work after the main site is live.
- **Uncertainty display:** show the probability and score as ranges, not single numbers, when inputs are estimated.

What to include in the README and report

1. Problem statement and what the site does, with screenshots or a short GIF.
2. Data sources, download date, and cleaning steps.
3. Model choices, metrics, and the leakage precautions you took.
4. The exact habitability formula, weights and thresholds, with the reasoning.
5. Limitations and ethical notes (what the scores do and do not mean).
6. How to run locally with Docker, and the link to the live demo.

References and resources

- NASA Exoplanet Archive: *exoplanetarchive.ipac.caltech.edu* (TAP service documentation and column definitions).
- Kopparapu et al. (2013), habitable zones around main-sequence stars, *The Astrophysical Journal*.
- Schulze-Makuch et al. (2011), a two-tiered approach to assessing the habitability of exoplanets (origin of the Earth Similarity Index), *Astrobiology*.
- Planetary Habitability Laboratory, Habitable Exoplanets Catalog (University of Puerto Rico at Arecibo): useful for validating your scores.
- Documentation for FastAPI, scikit-learn, XGBoost, SHAP, MLflow, React, Vite and Tailwind CSS.

Next step

Start with *scripts/fetch_data.py* and an EDA notebook on the Kepler table. Once the data is cached and a baseline model runs, everything else in this plan builds on that foundation.